

Proyecto N°2 de Estructuras de Datos

Videojuego de rol 'Fanfared Misadventures'

Integrantes:
Franco Rodriguez
Tomás Minte
Felipe Desmaras

Profesora: Jacqueline Aldridge

Ayudante: Milton Hernandez

Asignatura: Estructura de Datos

Fecha de entrega: 30 de octubre de 2026

Índice

1. Introducción **1**

2. Objetivos **1**

2.1. Objetivo General 1

2.2. Objetivos Especificos 1

3. Organizacion del proyecto **1**

3.1. Estructura del repositorio 1

3.2. Compilacion y ejecucion 3

3.3. Buenas practicas 3

4. Desarrollo e Implementacion del sistema **3**

4.1. Ciclo de vida del Sistema 3

4.2. Entidades Base del Juego 5

4.3. Inventario y Botín (TDA Lista Enlazada) 6

4.4. Sistema de Combate (TDA Cola Circular) 7

4.5. Historial y Deshacer Acción (TDA Pila) 8

4.6. Exploración y Persistencia de Partida 9

5. Pruebas, Validación y Manejo de Casos Límite **10**

5.1. Manejo de Casos Límite en las Estructuras de Datos 10

5.2. Validación de Entradas y Casos Borde de Juego 10

5.3. Ejemplo de Ejecución 11

6. Organización del Grupo **12**

7. Declaracion de IA **12**

8. Conclusion **13**

1. Introducción

El presente informe detalla el disenho y desarrollo de *Fanfared Misadventures*, un juego de rol **RPG** basado en texto, implementado en lenguaje C, en donde el jugador puede explorar distintas zonas, gestionar un inventario de objetos, enfrentarse a enemigos y registrar un historial. Toda la estructura de este proyecto se basa en la implementacion de tipos de datos abstractos lineales, especificamente, listas enlazadas, pilas y colas, y, por consiguiente, la implementación de la gestión de las estructuras dadas, cubriendo gestión de arreglos, creación, inserción, recorrido, eliminación, y búsqueda en listas enlazadas. El informe presenta el diseño y las estructuras utilizadas; después describe la generación y carga de datos, los algoritmos y las funcionalidades; finalmente documenta las pruebas, la gestión de memoria, las contribuciones, el uso de inteligencia artificial y las conclusiones.

2. Objetivos

2.1. Objetivo General

Disenhar e implementar un juego de rol RPG en lenguaje C, aplicando de manera correcta y justificada listas enlazadas, pilas y colas para gestionar personaje, enemigos, inventario, combate e historial.

2.2. Objetivos Especificos

- **Construir** y operar tipos de datos abstractos como las listas enlazadas, pilas y colas, asegurando que se cumplan cada uno de sus principios en cada mecanica del juego
- **Desarrollar** un modulo de exploracion interactivo y un sistema de combate que obligue al jugador a enfrentar grupos de enemigos siguiendo el orden de aparicion
- **Implementar** un historial de acciones basado en pilas
- **Gestionar** de manera precisa la memoria dinamica, controlando escenarios criticos como estructuras vacias o las fugas de memoria

3. Organizacion del proyecto

3.1. Estructura del repositorio

El codigo se divide en 7 modulos especializados en distintas responsabilidades ademas del programa principal *main.c*, evitando el uso de variables globales. Los modulos y su funcion son los siguientes:

Módulo (`inc/` y `src/`)	**Estructura / TDA Principal**	**Responsabilidad dentro del Sistema**
Programa principal (`main.c`)	Une todas las estructuras	Inicializa la semilla aleatoria, coordina la carga inicial desde `save/`, ejecuta el bucle principal (`loop_juego`) y asegura el guardado y liberación de memoria al salir
Interfaz y Menú (`game_menu.h` / `game_menu.c`)	Interfaz del juego	Controla la interacción por terminal, selección de partida, validación de entradas del usuario y navegación entre menus
Personaje (`personaje.h` / `personaje.c`)	`struct personaje_`	Administra los atributos del jugador (PV, ataque, defensa, objetos)
Objetos (`objetos.h` / `objeto.c`)	`struct objeto_`	Define las propiedades de los ítems (equipables, consumibles y accesorios), reserva dinámica de cadenas para nombre/descripción y actualización de estadísticas.
Inventario y Botín (`inventario.h` / `inv.c`)	`struct nodo_objeto`	Administra el inventario del jugador y las listas de botín por zona (`botin_bosque`, `botin_cuevas`, `botin_castillo`) a partir de `lootpool.csv`, además de la reversión del descarte de objetos.
Combate y Enemigos (`enemigos.h` / `enemigos.c`)	`struct registro_cola`	Gestiona la aparición secuencial de enemigos según la dificultad de la zona y ejecuta el combate por turnos respetando el orden FIFO.
Historial (`historial.h` / `historial.c`)	`struct nodo`	Registra cronológicamente las acciones de la partida bajo el principio LIFO, permitiendo visualizar los eventos recientes, deshacer el último descarte y guardar el registro en `historial.txt`.
Exploración (`exploracion.h` / `exploracion.c`)	Caminos, enemigos y objetos	Conecta al personaje con las tres zonas del juego, simulando el avance paso a paso y disparando eventos al azar de encontrar un botín o aparicion de enemigos.

Tabla 1: Diseño modular y distribución de responsabilidades.

3.2. Compilacion y ejecucion

Para automatizar la construccion del proyecto en un entorno Linux, se disenho e implemento un archivo `Makefile` que gestiona la compilacion de los modulos, especificamente transforma cada archivo `.c` de `src/` en un archivo `.o` dentro de `obj/`.
Entre algunas decisiones tecnicas de nuestro `Makefile` que decidimos utilizar, destacan el uso de las banderas de `-Wall Wextra -Wpedantic` para activar las advertencias del compilador y mediante `wildcard` y `patsubst` cualquier nuevo modulo agregado en `src/` se incorpora automaticamente al flujo de compilacion sin problemas

3.3. Buenas practicas

Se definio un estandar de trabajo en el README con el objetivo de mantener una legibilidad y estructura en toda la creacion del programa

Nomenclatura: snake_case (Para todo tipo de datos) Idioma: Español

- No subir a github codigo que no compile.
- Commits claros y descriptivos.
- Si una función reserva memoria, dejen claro quién o que función se encarga de destruirla para evitar fugas de memoria.
- Documentar las funciones en el archivo .h con doxygen (Estilo Milton en las prácticas)

4. Desarrollo e Implementacion del sistema

4.1. Ciclo de vida del Sistema

El archivo `src/main.c` actua como el punto de entrada y llama a las distintas funciones que manejan el sistema. Dado que el proyecto prescinde de variables globales, todas las estructuras principales de partida (registro del personaje, pila de historial y la lista enlazada del inventario y las tres listas de botin por zona) se declaran en el ambito local de `main()`.El orden de ejecución es el siguiente:

```
int main(void)
{
    srand(time(NULL));

    // 1. Declaracion e inicializacion de estructuras base
    personaje_ personaje = {"\0", 20, 2, 1, 0, 0, 0, crear_pila()};
    inventario i = NULL;
    inventario botin_bosque = NULL, botin_cuevas = NULL, botin_castillo =
   NULL;
```

```c
    int loop_juego = 1, indice_archivo_guardado = -1;
    // Seleccion de ranura (save_1, save_2, save_3) y reconstruccion de datos
    inicializacion_rpg(&indice_archivo_guardado);
    cargar_personaje(&personaje, &indice_archivo_guardado);
    i = cargar_inventario(i, indice_archivo_guardado);
    botin_bosque = cargar_botin(botin_bosque, 1, indice_archivo_guardado);
    botin_cuevas = cargar_botin(botin_cuevas, 2, indice_archivo_guardado);
    botin_castillo = cargar_botin(botin_castillo, 3, indice_archivo_guardado);
    cargar_historial(personaje.historial, &indice_archivo_guardado);
    if (archivo_guardado_vacio(&indice_archivo_guardado) == 1)
    {
        creacion_personaje(&personaje);
    }
    // 2. Bucle principal del juego (Game Loop)
    while(loop_juego)
    {
        imprimir_menu();
        selector_menu(&personaje, &loop_juego, i, botin_bosque,
                      botin_cuevas, botin_castillo, personaje.historial,
                      indice_archivo_guardado);
    }
    // 3. Guardado final y liberacion de memoria
    guardar_personaje(&personaje, &indice_archivo_guardado);
    guardar_inventario(i, indice_archivo_guardado);
    guardar_historial(personaje.historial, &indice_archivo_guardado);
    eliminar_inventario(i);
    return 0;
}
```

Listing 1: Estructura del ciclo de vida del programa en `src/main.c`.

1. **Etapa de Inicialización y guardado**
   - Se inicializa la semilla de números aleatorios (`srand(time(NULL))`) para los eventos de exploración y aparición de enemigos.
   - Se crea la estructura `personaje` con sus estadísticas base (20 PV, 2 de ataque, 1 de defensa) y se inicializa su pila de historial mediante `crear_pila()`.
   - Mediante `inicializacion_rpg()`, el jugador elige una de las 3 ranuras disponibles (`save_1`, `save_2` o `save_3`). A partir de ese índice, el programa reconstruye en memoria dinámica los atributos del jugador (`save.txt`), su inventario (`inventario.txt`), las listas de objetos disponibles en cada zona filtrando los ya agarrados (`objetos_tomados.txt` cruzado con `lootpool.csv`) y la pila de acciones previas (`historial.txt`).
   - Si la ranura está vacía (`archivo_guardado_vacio()` devuelve `1`), se solicita el nombre del personaje mediante `creacion_personaje()`.
2. **Etapa de Bucle Principal (*Game Loop*):**

- Controlada por la bandera `loop_juego`, esta fase despliega el menú principal y pasa el control a `selector_menu()`.
- el jugador puede: **(1)** explorar una de las tres zonas (Bosque de Almas, Sistema de Cuevas o Castillo en Ruinas), **(2)** inspeccionar las estadísticas actuales de su personaje, **(3)** listar su inventario, descartar objetos por ID o deshacer el último descarte, **(4)** revisar el historial de acciones, o **(5)** salir del juego (modificando `*loop_juego = 0`).

3. **Etapa de Cierre, Guardado y Limpieza:**

- Una vez que el usuario decide salir del bucle principal, el programa vuelca automáticamente el estado final del personaje, los identificadores de los objetos en su inventario y el registro completo de la pila de historial hacia los archivos `.txt` de la ranura correspondiente.
- Finalmente, se invoca `eliminar_inventario(i)` para recorrer los nodos de la lista enlazada y liberar la memoria dinámica reservada antes de retornar `0` al sistema operativo.

4.2. Entidades Base del Juego

Las entidades principales se definen mediante tres `struct` en sus respectivas cabeceras:

```
typedef struct {
    char nombre[20];
    int puntos_de_vida, ataque, defensa;
    int accesorio, regeneracion;
    bool espejo_activo;
    historial historial; // Pila asociada al jugador
} personaje_;

typedef struct {
    int id, clase;       // 1: EQUIPABLE, 2: CONSUMIBLE, 3: ACCESORIO
    char *nombre, *descripcion;
} objeto_;

typedef struct {
    int id;              // Orden de aparicion
    char nombre[20];
    int puntos_de_vida, ataque;
    bool envenenado;
} enemigo_;
```

Listing 2: Estructuras base en `personaje.h`, `objetos.h` y `enemigos.h`.

- **Uso de memoria en cadenas:** En `personaje_` y `enemigo_` se usa un arreglo fijo (`nombre[20]`) para acotar el largo en pantalla. En `objeto_` se usan punteros dinámicos (`char*`) reservados con `malloc(strlen(...) + 1)` en `crear_objeto()`, ya que los textos en `lootpool.csv` tienen tamaños variables.

- **Atributos adicionales:** Se agregaron `regeneracion`, `espejo_activo`, `accesorio` y `envenenado` para aplicar los efectos de pociones y accesorios en combate, además de anidar la pila `historial` dentro del personaje.

4.3. Inventario y Botín (TDA Lista Enlazada)

Justificación de la estructura: Se implementó una **Lista Simplemente Enlazada con nodo centinela** para el inventario y los botines de cada zona (`botin_bosque`, `botin_cuevas`, `botin_castillo`).

- Utilizamos una Lista Enlazada ya que el jugador necesita buscar objetos por nombre y usar o descartar cualquier ítem por su `id` sin importar en qué orden lo recogió. Una **Pila (LIFO)** solo permitiría usar el último objeto guardado y una **Cola (FIFO)** solo el más antiguo, impidiendo el acceso libre a la mochila sin romper las reglas de su TDA.

Operaciones implementadas (`src/inv.c`): La lista utiliza un nodo cabecera vacío creado en `crear_inventario_vacio()`, por lo que el primer objeto real está en `i->siguiente`.

- **Inserción (`insertar_objeto`):** Recorre la lista hasta el último nodo (`p->siguiente == NULL`), reserva memoria con `malloc` e inserta el nuevo objeto al final.
- **Búsqueda (`encontrar_objeto_por_id` / `_por_nombre`):** Recorre desde `i->siguiente` comparando el `id` o el nombre con `strcmp()`, retornando el puntero al nodo.
- **Eliminación (`eliminar_objeto_por_id`):** Usa `encontrar_objeto_anterior_por_id()` para ubicarse en el nodo previo (`p`), reenlaza el puntero saltándose el nodo objetivo (`p->siguiente = aux->siguiente`) y libera su memoria con `free(aux)`.

```
1  inventario insertar_objeto(inventario i, objeto_ o) {
2      if(i == NULL) return i;
3      posicion p = i;
4      while(p->siguiente != NULL) p = p->siguiente;
5
6      inventario aux = malloc(sizeof(struct nodo_objeto));
7      if(aux == NULL) return NULL;
8
9      aux->objeto = o;
10     aux->siguiente = NULL;
11     p->siguiente = aux;
12     return i;
13 }
14
15 void eliminar_objeto_por_id(inventario i, int id) {
16     if(inventario_vacio(i) == true) return;
17     posicion p = encontrar_objeto_anterior_por_id(i, id);
18     if(p == NULL) return;
19
```

```
    posicion aux = p->siguiente;
    p->siguiente = aux->siguiente; // Reenlaza saltando el nodo eliminado
    free(aux);
}
```

Listing 3: Inserción y eliminación en la lista enlazada (`src/inv.c`).

4.4. Sistema de Combate (TDA Cola Circular)

Justificación de la estructura: Para enfrentar grupos de hasta 3 enemigos se implementó una **Cola Circular (FIFO)** sobre un arreglo dinámico (`struct registro_cola`).

- Como queremos enfrentar a los enemigos estrictamente según su número de aparición (1, luego 2, luego 3), el TAD mas correcto es la Cola. Gracias al principio **FIFO (*First-In, First-Out*)** este nos asegura que el primer enemigo generado sea el primero en ser enfrentado. Una **Pila (LIFO)** invertiría el orden (peleando primero contra el enemigo 3), y una **Lista** añadiría punteros innecesarios cuando solo se requiere insertar al final y extraer del frente.

Operaciones implementadas (`src/enemigos.c`): La cola maneja los índices `front`, `rear`, `tamanho` y `capacidad`, avanzando de forma circular mediante la función `sucesor()`.

- `enqueue`**:** Al iniciar un combate (`enemigos_x_zona`), genera entre 1 y 3 enemigos según la zona y los inserta en `Q->rear`.
- **Acceso al frente:** Durante el turno (`ataque`), solo se interactúa con el enemigo en el frente (`&Q->arreglo[Q->front]`).
- `dequeue`**:** Cuando la vida del enemigo actual llega a `0`, se retira del frente avanzando `Q->front = sucesor(Q->front, Q)` y pasa el siguiente enemigo en espera.

```
int sucesor(int valor, cola_enemigos Q) {
    if(++valor == Q->capacidad) valor = 0;
    return valor;
}

cola_enemigos enqueue(enemigo_ X, cola_enemigos Q) {
    if(!cola_llena(Q)) {
        Q->tamanho++;
        Q->rear = sucesor(Q->rear, Q);
        Q->arreglo[Q->rear] = X;
    }
    return Q;
}

cola_enemigos dequeue(cola_enemigos Q) {
    if(!cola_vacia(Q)) {
```

```
        Q->tamanho--;
        Q->front = sucesor(Q->front, Q);
    }
    return Q;
}
```

Listing 4: Operaciones FIFO de la cola circular (`src/enemigos.c`).

4.5. Historial y Deshacer Acción (TDA Pila)

Justificación de la estructura: El historial se implementó como una **Pila (LIFO)** con nodos enlazados y nodo centinela.

- Para poder implementar correctamente el historial utilizamos una Pila, nuestro sistema debe permitir deshacer **únicamente la última acción** si esta fue descartar un objeto. Bajo el principio **LIFO (*Last-In, First-Out*)**, la última acción registrada queda siempre en el tope (`H->siguiente`), permitiendo verificarla y revertirla. Una **Cola** apuntaría a la acción más antigua de la partida, y una **Lista** requeriría recorrer toda la estructura hasta el final para deshacer.

Operaciones implementadas (`src/historial.c` y `src/inv.c`):

- `push` y `pop`: Insertan y eliminan acciones únicamente en el tope de la pila (`H->siguiente`), reservando y liberando la memoria del texto y del nodo.
- **Deshacer descarte (`deshacer_descarte_objeto`):** Revisa el texto en el tope (`H->siguiente->acci`) con `sscanf()`. Si coincide con `."El iminaste el objeto de id: %d"`, busca ese ID en `lootpool.csv`, lo devuelve al inventario con `insertar_objeto()` y elimina la acción con `pop(H)`. Si la última acción fue otra (ej. caminar o atacar), no coincide y no permite deshacer.
- **Impresión y guardado recursivo:** Las funciones `imprimir_desde_el_fondo()` y `escribir_desde_el_fondo()` usan recursividad para leer desde la base hasta el tope sin desarmar la pila.

```
historial push(historial H, char *accion_texto) {
    historial celda = malloc(sizeof(struct nodo));
    if (celda == NULL) return H;
    celda->accion = malloc(strlen(accion_texto) + 1);
    strcpy(celda->accion, accion_texto);

    celda->siguiente = H->siguiente; // Inserta en el tope
    H->siguiente = celda;
    return H;
}

historial pop(historial H) {
```

```
    if (H->siguiente == NULL) return H;
    historial primera = H->siguiente;
    H->siguiente = H->siguiente->siguiente; // Desapila el tope
    free(primera->accion);
    free(primera);
    return H;
}
```

Listing 5: Operaciones `push`, `pop` y reversión de descarte.

4.6. Exploración y Persistencia de Partida

- **Exploración (`src/exploracion.c`):** En `explorar_zona()`, el jugador avanza paso a paso generando un evento aleatorio (`rand() % 3`): **0** sigue caminando (se registra en la pila de historial), **1** extrae un objeto al azar de la lista de botín de la zona (`tomar_objeto_de_botin`) y **2** inicia un combate encolando enemigos de esa zona (`enemigos_x_zona`).
- **Guardado y Carga (`save_1`, `save_2`, `save_3`):** Cada partida guarda 4 archivos: `save.txt` (atributos del personaje), `historial.txt` (acciones de la pila) e `inventario.txt` / `objetos_tomados.txt` (que guardan solo los IDs de los objetos y al cargar se cruzan con `lootpool.csv` para reconstruir las listas enlazadas).

5. Pruebas, Validación y Manejo de Casos Límite

5.1. Manejo de Casos Límite en las Estructuras de Datos

En la Tabla 2 se detalla cómo cada Tipo de Dato Abstracto (TDA) implementado resuelve los cuatro casos límite fundamentales exigidos en el diseño del sistema:

Caso Límite	**Lista Enlazada** (`inventario` / `inv.c`)	**Cola Circular** (`cola_enemigos` / `enemigos.c`)	**Pila** (`historial` / `historial.c`)
1. Estructura Vacía	`inventario_vacio(i)` evalúa `i == NULL || i->siguiente == NULL`. Cancela búsquedas, usos o borrados e imprime `"Vacio!"`.	`cola_vacia(Q)` evalúa `Q->tamanho == 0`. Bloquea `dequeue()` sin enemigos y finaliza el bucle de combate al derrotar al último.	`pila_esta_vacia(H)` evalúa `H->siguiente == NULL`. Evita hacer `pop()` o deshacer descartes cuando no hay acciones.
2. Un Solo Elemento	El único ítem apunta a `NULL`. Al borrarlo, el nodo centinela (`i->siguiente`) pasa a `NULL` sin perder la cabecera de la lista.	`front` y `rear` apuntan al mismo índice (`tamanho == 1`). Al hacer `dequeue()`, `tamanho` pasa a `0` y `front` avanza con `sucesor()`.	El único nodo está en `H->siguiente`. Al ejecutar `pop()`, el centinela se actualiza a `NULL` (`H->siguiente = NULL`).
3. Memoria Insuficiente	Verifica `malloc == NULL` al crear nodos en `insertar_objeto()` y cadenas en `crear_objeto()` (liberando `nombre` si falla `descripcion`).	En `crear_cola()`, verifica `Q == NULL` y luego `Q->arreglo == NULL`. Si falla el arreglo, ejecuta `free(Q)` antes de retornar `NULL`.	En `push()`, comprueba `celda_temporal == NULL` antes de reservar la cadena dinámica `accion` y enlazar el nodo.
4. Eliminación y Liberación	`eliminar_objeto_por_id()` ubica el nodo anterior, reenlaza `p->siguiente = aux->siguiente` y hace `free(aux)`. Al salir, `eliminar_inventario()` libera toda la lista.	`dequeue()` realiza borrado lógico en $\mathcal{O}(1)$ decrementando `tamanho` y avanzando `front` circularmente sobre el arreglo reservado.	`pop()` desenlaza el tope y ejecuta doble liberación: primero el texto (`free(primera->accion)`) y luego el nodo (`free(primera)`).

Tabla 2: Manejo de casos límite en las tres estructuras de datos implementadas.

5.2. Validación de Entradas y Casos Borde de Juego

Adicionalmente, se controlaron las entradas inválidas del usuario y límites numéricos durante la partida:

- **Entradas por terminal:** `elegir_opcion()` valida que la entrada sea numérica (`scanf("%d",`

`&opcion) == 1`) y los menús rechazan valores fuera de rango. En `tomar_objeto_de_botin()`, solo se aceptan los caracteres `'S'` o `ñ'`.

- **Acotamiento de cadenas:** `creacion_personaje()` lee el nombre con `fgets(p->nombre, sizeof(p->nombre), stdin)`, evitando desbordamientos mayores a 19 caracteres.
- **Límites en combate:** En `ataque()`, si la defensa supera el ataque enemigo (`danho_enemigo <0`), el daño se trunca a `0`. Asimismo, los puntos de vida nunca bajan de `0`.

5.3. Ejemplo de Ejecución

El siguiente fragmento de terminal muestra un flujo completo: hallazgo de objeto en exploración, combate contra la cola de enemigos, descarte en el inventario y recuperación mediante la pila de historial:

```
====(Fanfared Misadventures)====
    1. Explorar zona | 2. Ver personaje | 3. Inventario | 4. Historial |
   5. Salir :(
Elige una opcion: 1
1. Para ir a Bosque de Almas -> Elige una opcion: 1
caminando...
Encontraste un objeto!
  - id: 1 | Nombre: Espada oxidada | Descripcion: Aumenta el ataque a 8.
Tomar Espada oxidada? (S/n) -> S

Deseas seguir explorando? (1: Seguir / 2: Volver al menu) -> 1
caminando...
Encontraste a unos enemigos!
  1. Duende (10 PV)
  2. Lobo (15 PV)
    ==== COMBATE ====
Heroe: 20 PV | Ataque 2 | Defensa 1
Enemigo actual: Duende (10 PV) | Enemigos en espera: 1
1. Atacar | 2. Ver inventario | 3. Usar objeto -> Elige una opcion: 1
...
Mataste a Duende. EPICO

# --- En el menu de Inventario (Opcion 3): Descartar y Deshacer ---
 Desea  descartar algun objeto? (1) Si | (2) No | (3) Recuperar objeto
   descartado
Elige una opcion: 1 -> Introduzca el id del objeto a eliminar: 1
Eliminando [Espada oxidada]...

# --- Al volver a entrar y seleccionar (3) Recuperar objeto descartado ---
Elige una opcion: 3
Objeto recuperado!
```

Listing 6: Traza de ejecución del sistema en terminal.

6. Organización del Grupo

El trabajo se dividió de forma equitativa asegurando que cada integrante implementara al menos un Tipos de Datos Abstractos (TDA). En la Tabla 3 se detalla el aporte individual al código fuente:

Integrante	**TDA Implementado / Utilizado**	**Módulos y Funcionalidades Desarrolladas**
Tomás Minte	**TDA Cola Circular** (`cola_enemigos`) y uso de **TDA Pila** (`push` en combate y exploración)	▪ Implementación del TDA Cola Circular (`crear_cola`, `enqueue`, `dequeue`, `sucesor`, `vaciar_cola`) en `enemigos.c`. ▪ Creación de enemigos por zona (`enemigos_bosque`, `_cuevas`, `_castillo`) y lógica del combate por turnos (`imprimir_combate`, `ataque`, condiciones de victoria y muerte). ▪ Sistema de elección de rutas y exploración con eventos probabilísticos (`camino`, `explorar_zona`) y funciones de navegación (`elegir_opcion`, `volver_menu`).
Felipe Desmaras	**TDA Lista Enlazada** (`inventario` y listas de botín por zona)	▪ Implementación del TDA Lista Enlazada con centinela (`crear_inventario_vacio`, `insertar_objeto`, búsquedas por ID y nombre, `eliminar_objeto_por_id`, `eliminar_inventario`) en `inv.c`. ▪ Diseño de la estructura `objeto_` (`crear_objeto`, `analizar_objeto`, `liberar_objeto`) y base de datos `lootpool.csv`. ▪ Lógica de botín por zonas (`cargar_botin`, `extraer_objeto_al_azar`, `tomar_objeto_de_botin`) y uso de ítems en combate (`usar_objeto_por_id`).
Franco Rodriguez	**TDA Pila** (`historial`) y operaciones sobre **TDA Lista Enlazada** (descarte y reversión)	▪ Implementación del TDA Pila (`crear_pila`, `push`, `pop`, `pila_esta_vacia`) y recorrido recursivo (`imprimir_desde_el_fondo`, `escribir_desde_el_fondo`) en `historial.c`. ▪ Implementación del módulo `personaje.c` y del sistema de persistencia multi-ranura (`cargar`/`guardar` personaje, historial e inventario en `save/`). ▪ Mecánica de descarte y recuperación del último objeto descartado (`eliminar_objeto_print`, `deshacer_descarte_objeto`) e integración general en `main.c` y `game_menu.c`.

Tabla 3: Distribución de tareas y aporte a las estructuras de datos por integrante.

7. Declaracion de IA

Franco Rodriguez: Uso de IA para debugear codigo en caso de errores de compilación y violaciones de segmento. Encontrar funciones propias de C y utilizarlas de manera apropiada en el codigo. Ej: rewind(). IAs de texto utilizadas 'claude', 'Gemini'.

8. Conclusion

El desarrollo del juego rpg Fanfared Misadventures permitió aplicar listas enlazadas, pilas y colas en distintos contextos de aplicacion, referente al juego, eligiendo cada estructura según las operaciones que exigía su funcionalidad.

La lista enlazada con nodo centinela resultó adecuada para el inventario y los botines por zona, ya que el jugador debe buscar, usar y descartar objetos por ID o nombre sin importar el orden en que los recogió. La cola circular aseguró, mediante su principio FIFO, que los enemigos se enfrentaran según su orden de aparición, con inserción y extracción en tiempo constante. La pila permitió mantener el historial con la acción más reciente siempre en el tope, lo que hizo posible revertir el último descarte de un objeto y rechazar la reversión cuando la última acción fue de otro tipo.

A grande rasgos el proyecto reforzó la comprensión del manejo de punteros y memoria dinámica en C, y la importancia de elegir una estructura de datos por sus propiedades. El trabajo en equipo con Git permitió que cada integrante implementara y usara al menos uno de los tipos de datos abstractos solicitados.